

Advanced GradleRIO

GradleRIO 高级配置

WPILib Docs 2026

中英对照 · Java

8 篇文档 · 按 WPILib 目录顺序汇编

Contents / 目录

Using External Libraries with Robot Code / 在机器人代码中使用外部库	3
Setting up CI for Robot Code using GitHub Actions / 使用 GitHub Actions 为机器人代码设置 CI	4
Using a Code Formatter / 使用代码格式化工具	9
Gradlew Tasks / Gradlew 任务	13
Including Git Data in Deploy / 在部署中包含 Git 数据	15
Using Compiler Arguments / 使用编译器参数	17
Profiling with VisualVM / 使用 VisualVM 进行性能分析	18
JVM Runtime error / JVM 运行时错误	27

Using External Libraries with Robot Code / 在机器人代码中使用外部库

Warning / 警告

Using external libraries may have unintended behavior with your robot code! It is not recommended unless you are aware of what you are doing!

使用外部库可能使机器人代码出现非预期行为！除非清楚自己在做什么，否则不建议这样做！

Often a team might want to add external Java libraries for usage with their robot code. This article highlights adding Java libraries to your Gradle dependencies.

队伍经常希望在机器人代码中使用外部 Java 库。本文重点介绍如何将 Java 库添加到 Gradle 依赖中。

Java / Java

Note / 说明

Any external dependencies that rely on native libraries (JNI) are likely not going to work.

任何依赖原生库（JNI）的外部依赖项都很可能无法正常工作。

Java is quite simple to add external dependencies. You simply add the required repositories and dependencies.

在 Java 中添加外部依赖相当简单。只需添加所需的 repositories 和 dependencies。

Robot projects by default do not have a repositories {} block in the build.gradle file. You will have to add this yourself. Above the dependencies {} block, please add the following:

机器人项目的 build.gradle 文件默认没有 repositories {} 代码块，需要自行添加。请在 dependencies {} 代码块上方添加以下内容：

```
repositories {
    mavenCentral()
    ...
}
```

mavenCentral() can be replaced with whatever repository the library you want to import is using. Now you have to add the dependency on the library itself. This is done by adding the necessary line to your dependencies {} block. The below example showcases adding Apache Commons to your Gradle project.

可以将 mavenCentral() 替换为要导入的库所使用的任意仓库。接下来，需要添加对库本身的依赖。这通过在 dependencies {} 代码块中添加必要的行来完成。以下示例展示如何将 Apache Commons 添加到 Gradle 项目。

```
dependencies {
    implementation 'org.apache.commons:commons-lang3:3.6'
    ...
}
```

Now you run a build and ensure the dependencies are downloaded. Intellisense may not work properly until a build is ran!

现在运行一次构建，确保依赖项已下载。在运行构建之前，Intellisense 可能无法正常工作！

Setting up CI for Robot Code using GitHub Actions / 使用 GitHub Actions 为机器人代码设置 CI

An important aspect of working in a team environment is being able to test code that is pushed to a central repository such as GitHub. For example, a project manager or lead developer might want to run a set of unit tests before merging a pull request or might want to ensure that all code on the main branch of a repository is in working order.

团队协作中的一个重要方面，是能够对推送到 GitHub 等中央仓库的代码进行测试。例如，项目经理或首席开发者可能希望在合并拉取请求之前运行一组单元测试，或确保仓库 main 分支上的全部代码都处于可正常工作的状态。

[GitHub Actions](#) is a service that allows for teams and individuals to build and run unit tests on code on various branches and on pull requests. These types of services are more commonly known as “Continuous Integration” services. This tutorial will show you how to setup GitHub Actions on robot code projects.

[GitHub Actions](#) 是一种服务，让队伍和个人能够构建各个分支及拉取请求中的代码，并对其运行单元测试。这类服务通常称为“持续集成”服务。本教程将展示如何在机器人代码项目中设置 GitHub Actions。

Note / 说明

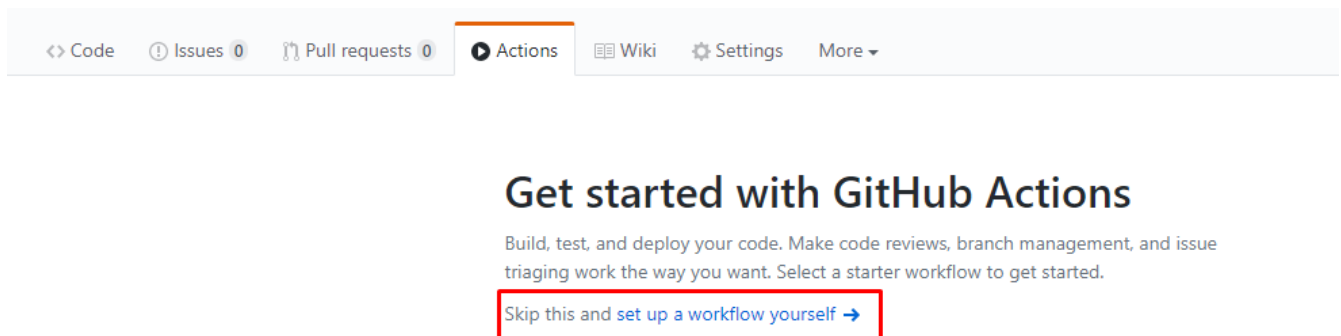
This tutorial assumes that your team’s robot code is being hosted on GitHub. For an introduction to Git and GitHub, please see this [introduction guide](#).

本教程假定队伍的机器人代码托管在 GitHub 上。有关 Git 和 GitHub 的介绍，请参阅这篇[入门指南](#)。

Creating the Action / 创建 Action

The instructions for carrying out the CI process are stored in a YAML file. To create this, click on the “Actions” tab at the top of your repository. Then click on the “set up a workflow yourself” hyperlink.

执行 CI 流程的指令保存在 YAML 文件中。要创建该文件，请点击仓库顶部的“Actions”选项卡，然后点击“set up a workflow yourself”超链接。



Setup a workflow yourself. / 自行设置工作流。

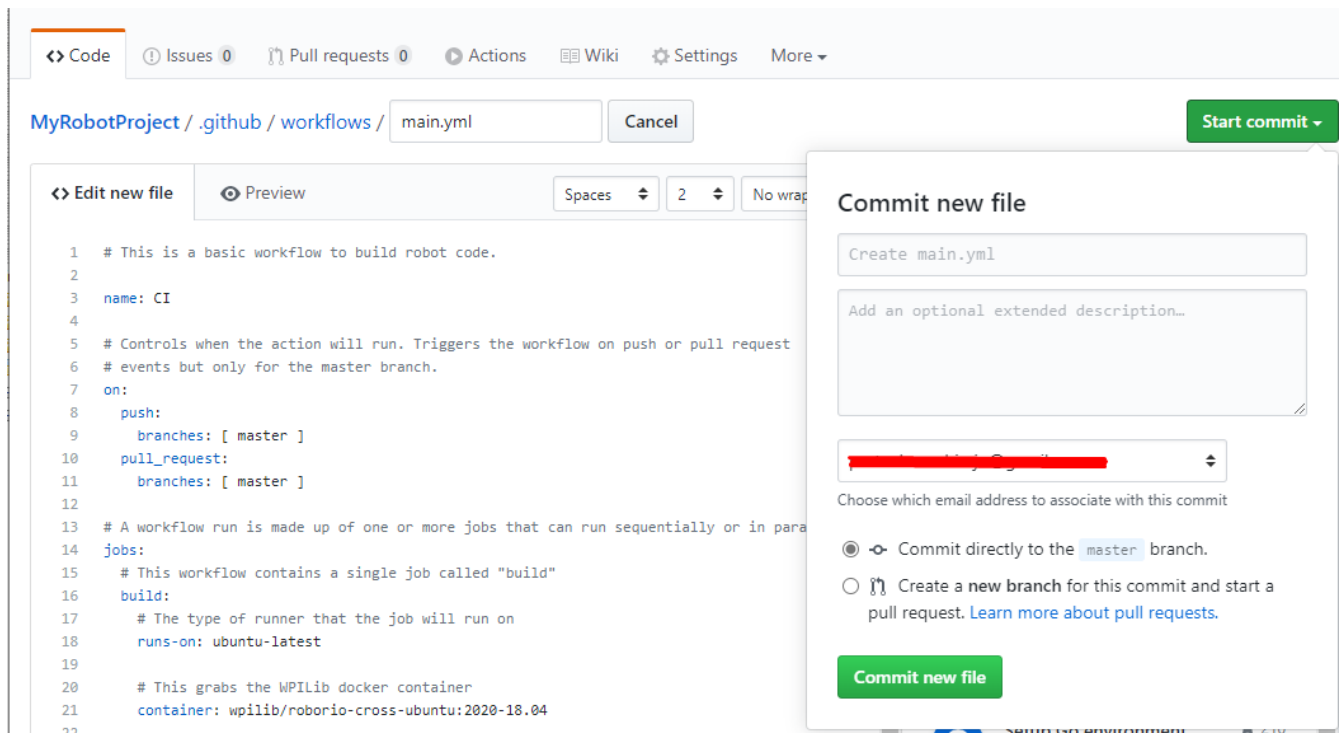
You will now be greeted with a text editor. Replace all the default text with the following:

接下来会出现一个文本编辑器。将全部默认文本替换为以下内容：

```
# This is a basic workflow to build robot code.
name: CI
# Controls when the action will run. Triggers the workflow on push or pull request
# events but only for the main branch.
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
# A workflow run is made up of one or more jobs that can run sequentially or in parallel
jobs:
  # This workflow contains a single job called "build"
  build:
    # The type of runner that the job will run on
    runs-on: ubuntu-22.04
    # This grabs the WPILib docker container
    container: wpilib/roborio-cross-ubuntu:2025-22.04
    # Steps represent a sequence of tasks that will be executed as part of the job
    steps:
      # Checks-out your repository under $GITHUB_WORKSPACE, so your job can access it
      - uses: actions/checkout@v4
      # Declares the repository safe and not under dubious ownership.
      - name: Add repository to git safe directories
        run: git config --global --add safe.directory $GITHUB_WORKSPACE
      # Grant execute permission for gradlew
      - name: Grant execute permission for gradlew
        run: chmod +x gradlew
      # Runs a single command using the runners shell
      - name: Compile and run tests on robot code
        run: ./gradlew build
```

Then, save changes by clicking the “Start commit” button on the top-right corner of the screen. You can amend the default commit message if you wish to do so. Then, click the green “Commit new file” button.

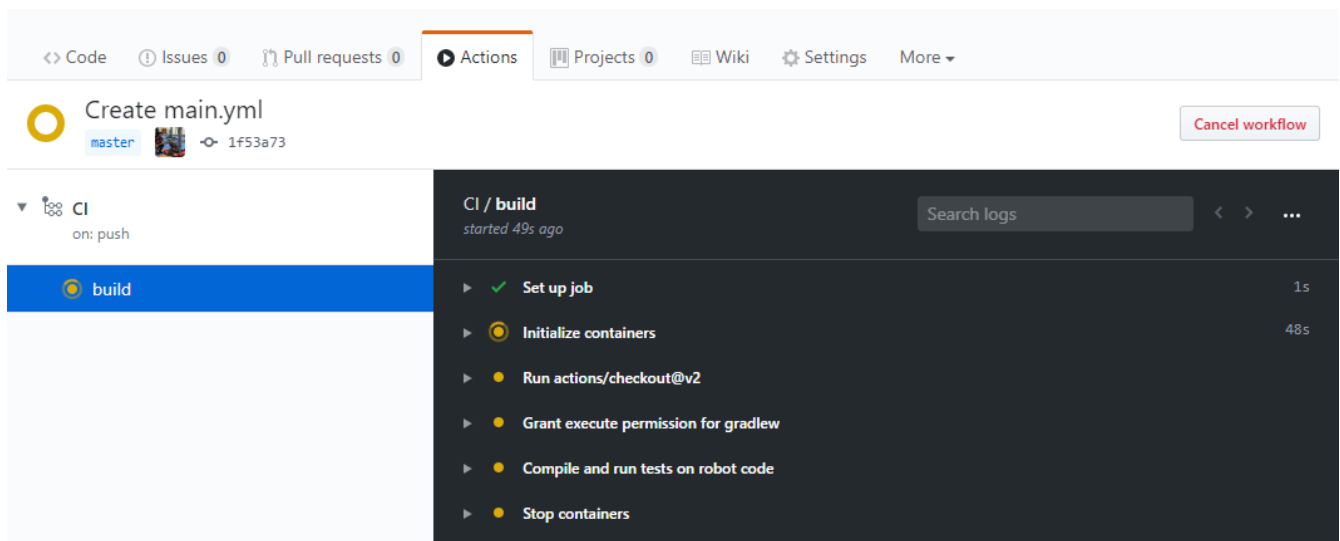
然后，点击屏幕右上角的“Start commit”按钮保存更改。如果愿意，可以修改默认提交消息。随后点击绿色的“Commit new file”按钮。



Committing a new file. / 提交新文件。

GitHub will now automatically run a build whenever a commit is pushed to main or a pull request is opened. To monitor the status of any build, you can click on the “Actions” tab on the top of the screen.

现在，每当向 main 推送提交或创建拉取请求时，GitHub 都会自动运行构建。要监控任意构建的状态，可以点击屏幕顶部的“Actions”选项卡。



View of the actions tab / Actions 选项卡视图

A Breakdown of the Actions YAML File / 拆解 Actions YAML 文件

Here is a breakdown of the YAML file above. Although a strict understanding of each line is not required, some level of understanding will help you add more features and debug potential issues that may arise.

以下对上述 YAML 文件进行拆解。虽然不要求严格理解每一行，但具备一定的理解有助于添加更多功能，以及调试可能出现的问题。

```
# Controls when the action will run. Triggers the workflow on push or pull request
# events but only for the main branch.
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

This block of code dictates when the Action will run. Currently, the action will run when commits are pushed to main or when pull requests are opened against main.

这段代码决定 Action 何时运行。目前，当向 main 推送提交，或创建以 main 为目标分支的拉取请求时，Action 就会运行。

```
# A workflow run is made up of one or more jobs that can run sequentially or in parallel
jobs:
  # This workflow contains a single job called "build"
  build:
    # The type of runner that the job will run on
    runs-on: ubuntu-22.04
    # This grabs the WPILib docker container
    container: wpilib/roborio-cross-ubuntu:2025-22.04
```

Each Action workflow is made of a one or more jobs that run either sequentially (one after another) or in parallel (at the same time). In our workflow, there is only one “build” job.

每个 Action 工作流由一个或多个任务组成，这些任务可以顺序执行（一个接一个），也可以并行执行（同时执行）。我们的工作流中只有一个名为“build”的任务。

We specify that we want the job to run on an Ubuntu virtual machine and in a virtualized [Docker container](#) that contains the JDK and roboRIO toolchains.

我们指定任务在 Ubuntu 虚拟机中运行，并在包含 JDK 和 roboRIO 工具链的虚拟化 [Docker](#) 容器中执行。

Note / 说明

Since none of the dependencies changed between 2025 and 2026, the 2025 docker images are still correct to use for 2026.

由于 2025 与 2026 之间没有任何依赖项发生变化，2025 的 Docker 镜像仍然可以正确用于 2026。

```
# Steps represent a sequence of tasks that will be executed as part of the job
steps:
# Checks-out your repository under $GITHUB_WORKSPACE, so your job can access it
- uses: actions/checkout@v4
# Declares the repository safe and not under dubious ownership.
- name: Add repository to git safe directories
  run: git config --global --add safe.directory $GITHUB_WORKSPACE
# Grant execute permission for gradlew
- name: Grant execute permission for gradlew
  run: chmod +x gradlew
# Runs a single command using the runners shell
- name: Compile and run tests on robot code
  run: ./gradlew build
```

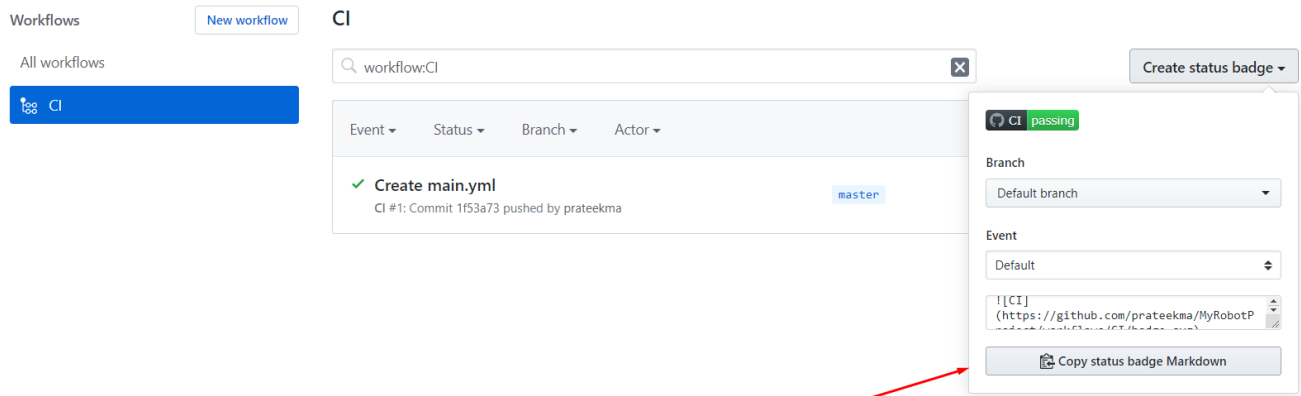
Each job has certain steps that will be executed. This job has four steps. The first step involves checking out the repository to access the robot code. The second step is a workaround for a [GitHub Actions issue](#). The third step involves giving the virtual machine permission to execute gradle tasks using `./gradlew`. The final step runs `./gradlew build` to compile robot code and run any unit tests.

每个任务都有一组要执行的步骤。此任务包含四个步骤。第一步检出仓库，以便访问机器人代码。第二步用于绕过一个 [GitHub Actions 问题](#)。第三步赋予虚拟机使用 `./gradlew` 执行 Gradle 任务的权限。最后一步运行 `./gradlew build`，编译机器人代码并运行所有单元测试。

Adding a Build Status Badge to a README.md File / 向 README.md 文件添加构建状态徽章

It is helpful to add a CI status badge to the top of your repository's README file to quickly check the status of the latest build on main. To do this, click on the "Actions" tab at the top of the screen and select the "CI" tab on the left side of the screen. Then, click on the "Create status badge" button on the top right and copy the status badge Markdown code.

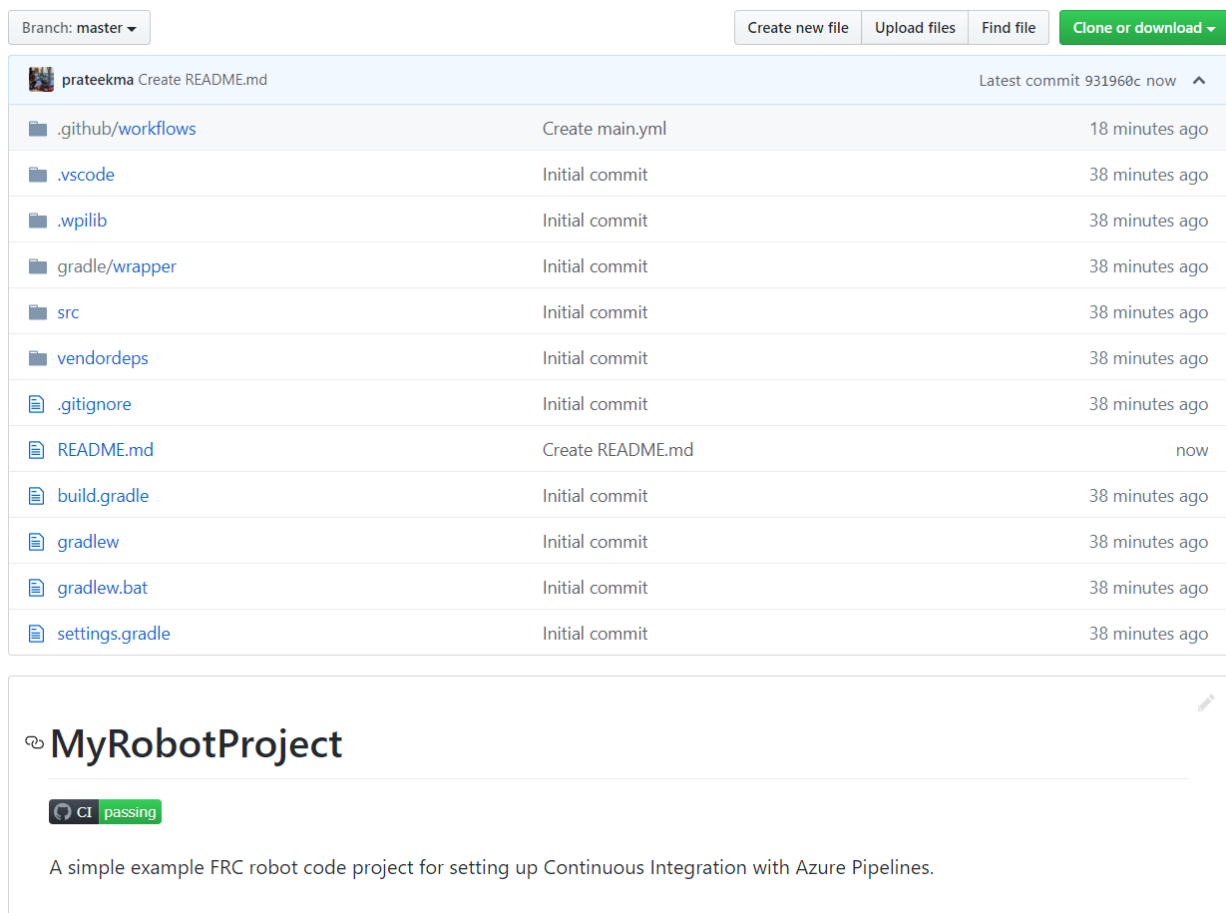
在仓库 README 文件顶部添加 CI 状态徽章，有助于快速查看 main 上最新构建的状态。为此，点击屏幕顶部的 "Actions" 选项卡，再选择屏幕左侧的 "CI" 选项卡。然后点击右上角的 "Create status badge" 按钮，复制状态徽章的 Markdown 代码。



Where to click on "Copy status badge Markdown" / "Copy status badge Markdown"的点击位置

Finally, paste the Markdown code you copied at the top of your README file, commit, and push your changes. Now, you should see the GitHub Actions status badge on your main repository page.

最后，将复制的 Markdown 代码粘贴到 README 文件顶部，提交并推送更改。现在，应该可以在仓库主页上看到 GitHub Actions 状态徽章。



A picture of the repository with the badge created. / 创建徽章后的仓库页面。

Using a Code Formatter / 使用代码格式化工具

Code formatters exist to ensure that the style of code written is consistent throughout the entire codebase. This is used in many major projects; from Android to OpenCV. Teams may wish to add a formatter throughout their robot code to ensure that the codebase maintains readability and consistency throughout.

代码格式化工具用于确保整个代码库中的代码风格一致。从 Android 到 OpenCV，许多大型项目都使用这类工具。队伍可能希望在整个机器人代码项目中加入格式化工具，以确保代码库始终保持可读性和一致性。

For this article, we will highlight using [Spotless](#) for Java teams.

本文将重点介绍 Java 队伍如何使用 [Spotless](#)。

Spotless / Spotless

Configuration / 配置

Necessary build.gradle changes are required to get Spotless functional. In the `plugins {}` block of your build.gradle, add the Spotless plugin so it appears similar to the below.

要使 Spotless 正常工作，需要对 build.gradle 作必要的修改。在 build.gradle 的 `plugins {}` 代码块中添加 Spotless 插件，使其类似于下方示例。

```
plugins {
    id "java"
    id "edu.wpi.first.GradleRIO" version "2024.1.1"
    id 'com.diffplug.spotless' version '6.20.0'
}
```

译注：本节保留了原文的 GradleRIO 2024.1.1 和 Spotless 6.20.0 示例版本，后文 CI 示例还使用 JDK 17；这些示例未更新为 2026 赛季配置。

Then ensure you add a required `spotless {}` block to correctly configure spotless. This can just get placed at the end of your build.gradle.

然后，务必添加所需的 `spotless {}` 代码块，以正确配置 spotless。该代码块可以直接放在 build.gradle 的末尾。

```
spotless {
    java {
        target fileTree('.') {
            include '**/*.java'
            exclude '**/build/**', '**/build-*/**', '**/bin/**'
        }
        toggleOffOn()
        googleJavaFormat()
        removeUnusedImports()
        trimTrailingWhitespace()
        endWithNewline()
    }
    groovyGradle {
        target fileTree('.') {
            include '**/*.gradle'
            exclude '**/build/**', '**/build-*/**'
        }
        greclipse()
        indentWithSpaces(4)
        trimTrailingWhitespace()
        endWithNewline()
    }
    format 'xml', {
        target fileTree('.') {
            include '**/*.xml'
        }
    }
}
```

```

        exclude '**/build/**', '**/build-*/**'
    }
    eclipseWtp('xml')
    trimTrailingWhitespace()
    indentWithSpaces(2)
    endWithNewline()
}
format 'misc', {
    target fileTree('.') {
        include '**/*.md', '**/.gitignore'
        exclude '**/build/**', '**/build-*/**'
    }
    trimTrailingWhitespace()
    indentWithSpaces(2)
    endWithNewline()
}
}

```

Running Spotless / 运行 Spotless

Spotless can be ran using `./gradlew spotlessApply` which will apply all formatting options. You can also specify a specific task by just adding the name of formatter. An example is `./gradlew spotlessmiscApply`.

可以使用 `./gradlew spotlessApply` 运行 Spotless，这会应用全部格式化选项。也可以通过添加格式化工具的名称来指定某个任务，例如 `./gradlew spotlessmiscApply`。

In addition to formatting code, Spotless can also ensure the code is correctly formatted; this can be used by running `./gradlew spotlessCheck`. Thus, Spotless can be used as a [CI check](#), as shown in the following GitHub Actions workflow:

除了格式化代码，Spotless 还可以通过运行 `./gradlew spotlessCheck` 来确保代码格式正确。因此，Spotless 可以用作 [CI 检查](#)，如下方 GitHub Actions 工作流所示：

```

on: [push]
# A workflow run is made up of one or more jobs that can run sequentially or in parallel
jobs:
  spotless:
    # The type of runner that the job will run on
    runs-on: ubuntu-22.04
    # Steps represent a sequence of tasks that will be executed as part of the job
    steps:
      # Checks-out your repository under $GITHUB_WORKSPACE, so your job can access it
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: actions/setup-java@v4
        with:
          distribution: 'zulu'
          java-version: 17
      - run: ./gradlew spotlessCheck

```

Explanation of Options / 选项说明

Each format section highlights formatting of custom files in the project. The java and groovyGradle are natively supported by spotless, so they are defined differently.

每个 format 部分用于指定项目中自定义文件的格式化方式。spotless 原生支持 java 和 groovyGradle，因此它们的定义方式不同。

Breaking this down, we can split this into multiple parts.

拆解这一配置，可以将其分为多个部分。

- Formatting Java

格式化 Java

- Formatting Gradle files

格式化 Gradle 文件

- Formatting XML files

格式化 XML 文件

- Formatting Miscellaneous files

格式化其他文件

They are all similar, except for some small differences that will be explained. The below example will highlight the `java {}` block.

这些部分都很相似，只有一些小差异，下面会进行说明。以下示例重点展示 `java {}` 代码块。

```
java {
    target fileTree('.') {
        include '**/*.java'
        exclude '**/build/**', '**/build-*/**'
    }
    toggleOffOn()
    googleJavaFormat()
    removeUnusedImports()
    trimTrailingWhitespace()
    endWithNewline()
}
```

Let's explain what each of the options mean.

下面解释各个选项的含义。

```
target fileTree('.') {
    include '**/*.java'
    exclude '**/build/**', '**/build-*/**'
}
```

The above example tells spotless where our Java classes are and to exclude the build directory. The rest of the options are fairly self-explanatory.

上面的示例告诉 spotless Java 类的位置，并排除 build 目录。其余选项的含义基本都很直观。

- `toggleOffOn()` adds the ability to have spotless ignore specific portions of a project. The usage looks like the following

`toggleOffOn()` 使 spotless 能够忽略项目中的特定部分。用法如下：

```
// format:off
public void myWeirdFunction() {
}
// format:on
```

- `googleJavaFormat()` tells spotless to format according to the [Google Style Guide](#)

`googleJavaFormat()` 告诉 spotless 按照 [Google 风格指南](#) 进行格式化

- `removeUnusedImports()` will remove any unused imports from any of your Java classes

`removeUnusedImports()` 会移除所有 Java 类中未使用的 import

- `trimTrailingWhitespace()` will remove any extra whitespace at the end of your lines

`trimTrailingWhitespace()` 会移除每行末尾多余的空白字符

- `endWithNewline()` will add a newline character to the end of your classes

`endWithNewline()` 会在类文件末尾添加一个换行字符

In the `groovyGradle` block, there is a `greclipse` option. This is the formatter that `spotless` uses to format gradle files.

`groovyGradle` 代码块中有一个 `greclipse` 选项。这是 `spotless` 用于格式化 Gradle 文件的格式化工具。

Additionally, there is a `eclipsewtp` option in the `xml` block. This stands for “Gradle Web Tools Platform” and is the formatter to format xml files. Teams not using any XML files may wish to not include this configuration.

此外，`xml` 代码块中有一个 `eclipsewtp` 选项。它代表“Gradle Web Tools Platform”，是用于格式化 xml 文件的格式化工具。不使用任何 XML 文件的队伍可以考虑不包含此配置。

译注：原文将 WTP 展开为“Gradle Web Tools Platform”；这里的 `eclipseWtp` 实际指 [Eclipse Web Tools Platform](#)。

Note / 说明

A full list of configurations is available on the [Spotless README](#)

完整配置列表可在 [Spotless README](#) 中找到

Issues with Line Endings / 行结束符问题

`Spotless` will attempt to apply line endings per-OS, which means Git diffs will be constantly changing if two users are on different OSes (Unix vs Windows). It's recommended that teams who contribute to the same repository from multiple OSes utilize a `.gitattributes` file. The following should suffice for handling line endings.

`Spotless` 会尝试按操作系统使用相应的行结束符。这意味着，如果两名用户使用不同的操作系统（Unix 与 Windows），Git diff 就会不断变化。建议使用多种操作系统向同一仓库贡献代码的队伍使用 `.gitattributes` 文件。以下配置应足以处理行结束符。

```
*.gradle text eol=lf
*.java text eol=lf
*.md text eol=lf
*.xml text eol=lf
```

Gradle Tasks / Gradlew 任务

This article aims to highlight the gradle commands supported by the WPILib team for user use. These commands can be viewed by typing `./gradlew tasks` at the root of your robot project. Not all commands shown in `./gradlew tasks` and unsupported commands will not be documented here.

本文旨在介绍 WPILib 团队支持用户使用的 Gradle 命令。在机器人项目根目录中输入 `./gradlew tasks` 即可查看这些命令。本文不会列出 `./gradlew tasks` 显示的所有命令，也不会介绍不受支持的命令。

Build tasks / 构建任务

`./gradlew build` - Assembles and tests this project. Useful for prebuilding your project without deploying to the roboRIO.

`./gradlew build` - 组装并测试本项目。适合在不部署到 roboRIO 的情况下预先构建项目。

`./gradlew clean` - Deletes the build directory.

`./gradlew clean` - 删除构建目录。

DeployUtils tasks / DeployUtils 任务

`./gradlew deploy` - Deploy all artifacts on all targets. This will deploy your robot project to the available targets (IE, roboRIO).

`./gradlew deploy` - 在所有目标上部署所有构件。这会将机器人项目部署到可用目标（例如 roboRIO）。

`./gradlew discoverRoborio` - Determine the address(es) of target roboRIO. This will print out the IP address of a connected roboRIO.

`./gradlew discoverRoborio` - 确定目标 roboRIO 的地址。此命令会输出已连接的 roboRIO 的 IP 地址。

GradleRIO tasks / GradleRIO 任务

`./gradlew $TOOL$` - Runs the tool \$TOOL\$ (Replace \$TOOL\$ with the name of the tool. IE, Glass, Shuffleboard, etc)

`./gradlew $TOOL$` - 运行工具 \$TOOL\$（将 \$TOOL\$ 替换为工具名称，例如 Glass、Shuffleboard 等）

`./gradlew $TOOL$Install` - Installs the java tool \$TOOL\$ (Replace \$TOOL\$ with the name of the tool. IE, Shuffleboard, SmartDashboard, etc)

`./gradlew $TOOL$Install` - 安装 Java 工具 \$TOOL\$（将 \$TOOL\$ 替换为工具名称，例如 Shuffleboard、SmartDashboard 等）

`./gradlew InstallAllTools` - Installs all available tools. This excludes the development environment such as Visual Studio Code. It's the users requirement to ensure the required dependencies (Java) is installed. Only recommended for advanced users!

`./gradlew InstallAllTools` - 安装所有可用工具。其中不包含 Visual Studio Code 等开发环境。用户必须确保已安装所需依赖（Java）。仅建议高级用户使用！

`./gradlew simulateExternalJavaRelease` - Simulate External Task for Java/JVM. Exports a JSON file for use by editors / tools

`./gradlew simulateExternalJavaRelease` - 用于 Java/JVM 的外部仿真任务。导出 JSON 文件，供编辑器 / 工具使用

`./gradlew simulateJava` - Launches simulation for the Java projects

`./gradlew simulateJava` - 启动 Java 项目的仿真

`./gradlew vendordep` - Install vendordep JSON file from URL or local installation. See [3rd Party Libraries](#)

`./gradlew vendordep` - 从 URL 或本地安装位置安装 vendordep JSON 文件。请参阅[第三方库](#)

Including Git Data in Deploy / 在部署中包含 Git 数据

This article will explain how to include metadata from Git in robot code using the [gversion](#) Gradle plugin. This generates a file which can be used for accessing Git metadata in robot code. This can be used to track what revision of code is on the robot, such as by printing or logging it.

本文将介绍如何使用 [gversion](#) Gradle 插件，在机器人代码中包含 Git 元数据。此插件会生成一个文件，让机器人代码能够访问 Git 元数据。可以通过打印或记录这些信息，跟踪机器人上运行的是哪一个代码修订版本。

Installing gversion / 安装 gversion

To install gversion add the following line to the plugins block of build.gradle. This tells Gradle to use gversion in the project.

要安装 gversion，请将以下行添加到 build.gradle 的 plugins 代码块中。这会告诉 Gradle 在项目中使用 gversion。

```
plugins {
    // ...
    id "com.peterabeles.gversion" version "1.10"
}
```

In order to generate the file when building the project, add the following block to the bottom of build.gradle.

为了在构建项目时生成该文件，请将以下代码块添加到 build.gradle 的底部。

```
project.compileJava.dependsOn(createVersionFile)
gversion {
    srcDir      = "src/main/java/"
    classPackage = "frc.robot"
    className    = "BuildConstants"
    dateFormat   = "yyyy-MM-dd HH:mm:ss z"
    timeZone     = "America/New_York" // Use preferred time zone
    indent      = "  "
}
```

The srcDir, classPackage, and className parameters tell the plugin where to put the manifest file, and what to name it. The timeZone field can be set to your team's time zone based on [this list of timezone IDs](#). The dateFormat parameter is based on [this Java class](#).

srcDir、classPackage 和 className 参数告诉插件将清单文件放在哪里，以及使用什么名称。可以依据[此时区 ID 列表](#)，将 timeZone 字段设置为队伍所在的时区。dateFormat 参数基于[这个 Java 类](#)。

To test this, run a build of your project through the WPILib menu in the top right. Once the code has finished building, there should be a file called BuildConstants.java in your src/main/java folder. The following is an example of what this file should look like:

要验证这一配置，请通过右上角的 WPILib 菜单构建项目。代码构建完成后，src/main/java 文件夹中应出现一个名为 BuildConstants.java 的文件。下方展示该文件应有的示例内容：

```

package frc.robot;
/**
 * Automatically generated file containing build version information.
 */
public final class BuildConstants {
    public static final String MAVEN_GROUP = "";
    public static final String MAVEN_NAME = "GitVersionTest";
    public static final String VERSION = "unspecified";
    public static final int GIT_REVISION = 1;
    public static final String GIT_SHA = "fad108a4b1c1dcdcf8859c6295ea64e06d43f557";
    public static final String GIT_DATE = "2023-10-26 17:38:59 EDT";
    public static final String GIT_BRANCH = "main";
    public static final String BUILD_DATE = "2023-10-27 12:29:57 EDT";
    public static final long BUILD_UNIX_TIME = 1698424197122L;
    public static final int DIRTY = 0;
    private BuildConstants(){}
}

```

DIRTY indicates whether there are uncommitted changes in the project. A value of 0 indicates a clean repository, a value of 1 indicates that there are uncommitted changes, and a value -1 indicates an error.

DIRTY 表示项目中是否存在未提交的更改。值为 0 表示仓库干净，值为 1 表示存在未提交的更改，值为 -1 表示发生错误。

Ignoring Generated Files with Git / 让 Git 忽略生成的文件

These files are regenerated every time code is built or deployed, so it isn't necessary to track them with Git. The aptly named `.gitignore` file tells Git not to track any listed files and should exist by default in any project generated by the WPILib VS Code extension. Below is the line you should add to `.gitignore` to ignore the generated file:

这些文件会在每次构建或部署代码时重新生成，因此不必使用 Git 跟踪它们。顾名思义，`.gitignore` 文件会告诉 Git 不要跟踪其中列出的文件；由 WPILib VS Code 扩展生成的任何项目都应默认包含该文件。为忽略生成的文件，应将以下行添加到 `.gitignore`：

```
src/main/java/frc/robot/BuildConstants.java
```

Using Compiler Arguments / 使用编译器参数

Compiler arguments allow us to change the behavior of our compiler. This includes making warnings into errors, ignoring certain warnings and choosing optimization level. Normally it could be proposed that the solution is to pass them in as flags when compiling our code but this doesn't work in GradleRIO. Instead modify the build.gradle.

编译器参数可以改变编译器的行为，包括将警告视为错误、忽略特定警告，以及选择优化级别。通常可能会认为，解决方式是在编译代码时将这些参数作为标志传入，但这在 GradleRIO 中不起作用。应改为修改 build.gradle。

Warning / 警告

Modifying arguments is dangerous and can cause unexpected behavior.

修改参数具有危险性，可能导致非预期行为。

Java / Java

Compiler arguments can also be configured for Java. We do this by adding arguments in the JavaCompile task.

Java 也可以配置编译器参数。做法是在 JavaCompile 任务中添加参数。

```
// Configure string concat to always inline compile
tasks.withType(JavaCompile) {
    options.compilerArgs.add '-XDstringConcat=inline'
}
```

JVM Arguments / JVM 参数

Along with being able to configure compiler arguments Java also allows us to configure runtime options for the JVM. We do this by editing the frcJava artifact's arguments.

除了配置编译器参数，Java 还允许配置 JVM 的运行时选项。做法是编辑 frcJava 构件的参数。

```
frcJava(getArtifactClass('FRCJavaArtifact')) {
    jvmArgs.add("-XX:+DisableExplicitGC")
}
```

Profiling with VisualVM / 使用 VisualVM 进行性能分析

This document is intended to familiarize the reader with the diagnostic tool that is [VisualVM](#) for debugging Java robot programs. VisualVM is a tool for profiling JVM based applications, such as viewing why an application is using a large amount of memory. This document assumes the reader is familiar with the *risks* associated with modifying their robot build.gradle. This tutorial also assumes that the user knows basic terminal/commandline knowledge.

本文旨在让读者熟悉用于调试 Java 机器人程序的诊断工具 [VisualVM](#)。VisualVM 可以对基于 JVM 的应用进行性能分析，例如查看应用为何占用大量内存。本文假定读者熟悉修改机器人项目 build.gradle 所带来的风险。本教程还假定用户了解基本的终端 / 命令行知识。

Unpacking VisualVM / 解压 VisualVM

To begin, [download VisualVM](#) and unpack it to the WPILib installation folder. The folder is located at ~/wpilib/ where ~ indicates the users home directory. On Windows, this is C:\Users\Public\wpilib.

首先，[下载 VisualVM](#)，并将其解压到 WPILib 安装文件夹。该文件夹位于 ~/wpilib/，其中 ~ 表示用户的主目录。在 Windows 上，该位置为 C:\Users\Public\wpilib。

Setting up Gradle / 设置 Gradle

GradleRIO supports passing JVM launch arguments, and this is what is necessary to enable remote debugging. Remote debugging is a feature that allows a local machine (such as the user's desktop) to view important information about a remote target (in our case, a roboRIO). To begin, locate the frcJava code block located in the projects build.gradle. Below is what it looks like.

GradleRIO 支持传递 JVM 启动参数，这是启用远程调试所必需的。远程调试允许本地计算机（例如用户的桌面计算机）查看远程目标（这里是 roboRIO）的重要信息。首先，找到项目 build.gradle 中的 frcJava 代码块。其内容如下。

```

deploy {
    targets {
        roboRIO(getTargetTypeClass('RoboRIO')) {
            // Team number is loaded either from the .wpilib/wpilib_preferences.json
            // or from command line. If not found an exception will be thrown.
            // You can use getTeamOrDefault(team) instead of getTeamNumber if you
            // want to store a team number in this file.
            team = project.frc.getTeamNumber()
            debug = project.frc.getDebugOrDefault(false)

            artifacts {
                // First part is artifact name, 2nd is artifact type
                // getTargetTypeClass is a shortcut to get the class type using a string

                frcJava(getArtifactTypeClass('FRCJavaArtifact')) {
                }

                // Static files artifact
                frcStaticFileDeploy(getArtifactTypeClass('FileTreeArtifact')) {
                    files = project.fileTree('src/main/deploy')
                    directory = '/home/lvuser/deploy'
                    deleteOldFiles = false // Change to true to delete files on roboRIO that no
                                           // longer exist in deploy directory of this project
                }
            }
        }
    }
}

```

We will be replacing the highlighted lines with:

我们将把高亮显示的代码行替换为：

译注：原页面高亮的是上方 `frcJava(getArtifactTypeClass('FRCJavaArtifact'))` 的空代码块；Markdown 代码块未保留高亮样式。

```
frcJava(getArtifactTypeClass('FRCJavaArtifact')) {
    // Enable VisualVM connection
    jvmArgs.add("-Dcom.sun.management.jmxremote=true")
    jvmArgs.add("-Dcom.sun.management.jmxremote.port=1198")
    jvmArgs.add("-Dcom.sun.management.jmxremote.local.only=false")
    jvmArgs.add("-Dcom.sun.management.jmxremote.ssl=false")
    jvmArgs.add("-Dcom.sun.management.jmxremote.authenticate=false")
    jvmArgs.add("-Djava.rmi.server.hostname=10.TE.AM.2") // Replace TE.AM with team number
}
```

We are adding a few arguments here. In order:

这里添加了几个参数，依次为：

- Enable remote debugging

启用远程调试

- Set the remote debugging port to 1198

将远程调试端口设置为 1198

- Allow listening from remote targets

允许来自远程目标的连接

- Disable SSL authentication being required

取消必须进行 SSL 身份验证的要求

- Set the hostname to the roboRIOs team number. Be sure to replace this. ([TE.AM IP Notation](#))

根据 roboRIO 的队号设置主机名。务必替换此值。（[TE.AM IP 表示法](#)）

Important / 重要

The hostname when connected via USB-B should be 172.22.11.2.

通过 USB-B 连接时，主机名应为 172.22.11.2。

Running VisualVM / 运行 VisualVM

Launching VisualVM is done via the commandline with a few parameters. First, we navigate to the directory containing VisualVM. Then, launch it with parameters, passing it the WPILib JDK path. On a Windows machine, it looks like the following:

通过命令行并提供几个参数来启动 VisualVM。首先，进入包含 VisualVM 的目录。然后带参数启动 VisualVM，传入 WPILib JDK 路径。在 Windows 计算机上，命令如下：

```
cd "C:\Users\Public\wpilib\visualvm_2110\bin"
./visualvm --jdkhome "C:\Users\Public\wpilib\2026\jdk"
```

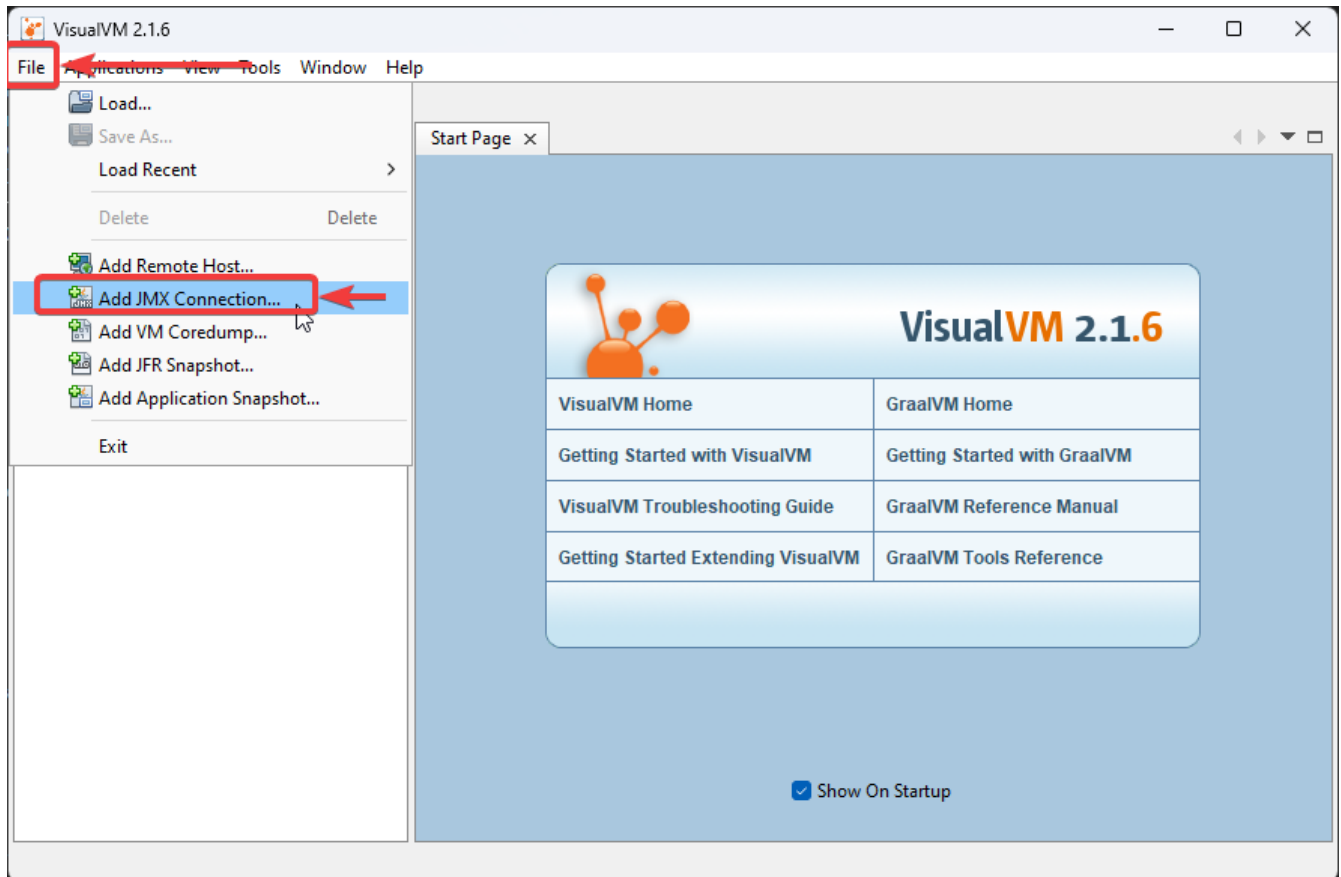
Important / 重要

The exact path `visualvm_2110` may vary and depends on the version of VisualVM downloaded.

具体路径 `visualvm_2110` 可能不同，取决于下载的 VisualVM 版本。

This should launch VisualVM. Once launched, open the Add JMX Connection dialog.

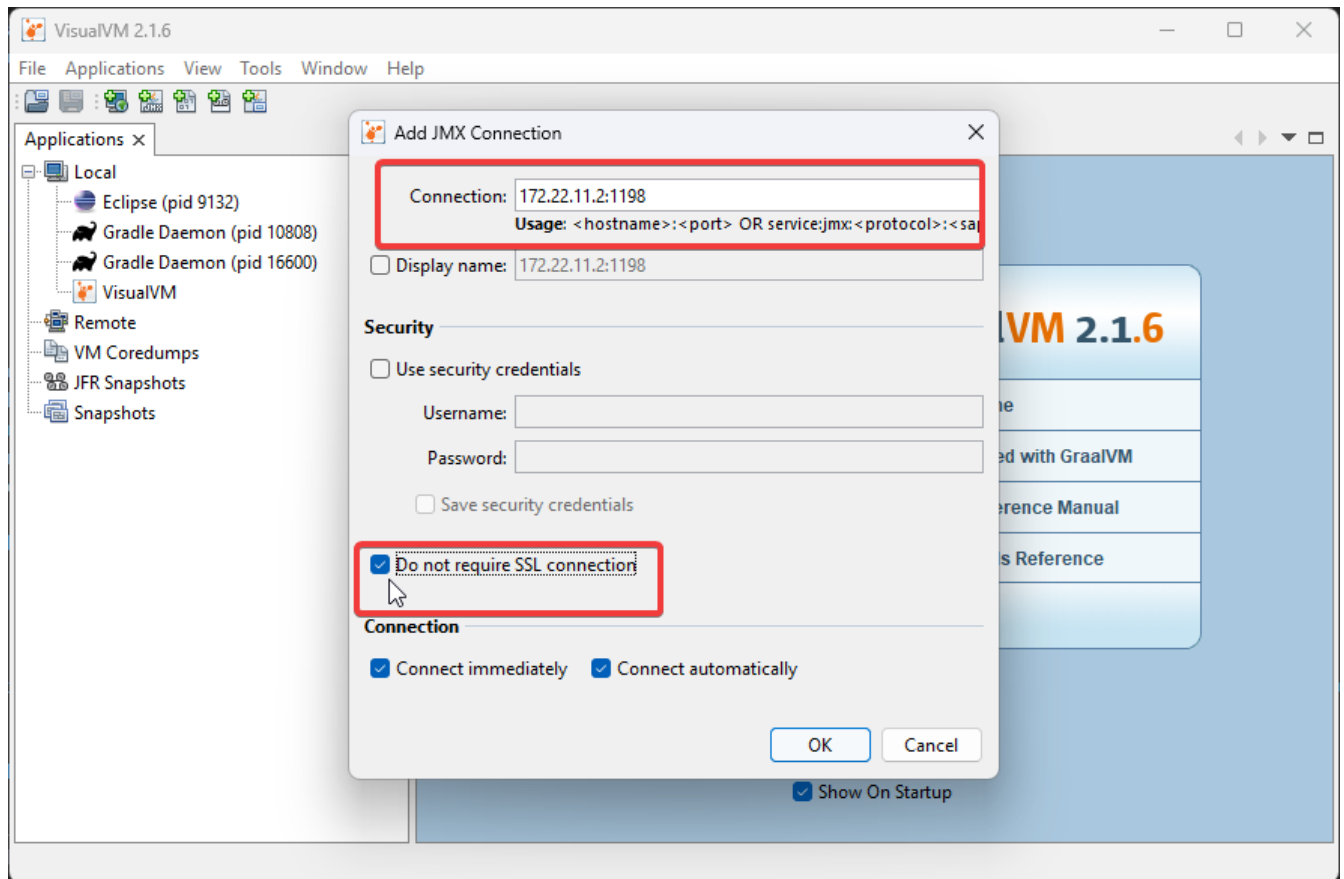
这应该会启动 VisualVM。启动后，打开 Add JMX Connection 对话框。



Add visualvm connection menu option / 添加 VisualVM 连接的菜单选项

Once opened, configure the connection details and hostname. Ensure that Do not require SSL connection is ticked.

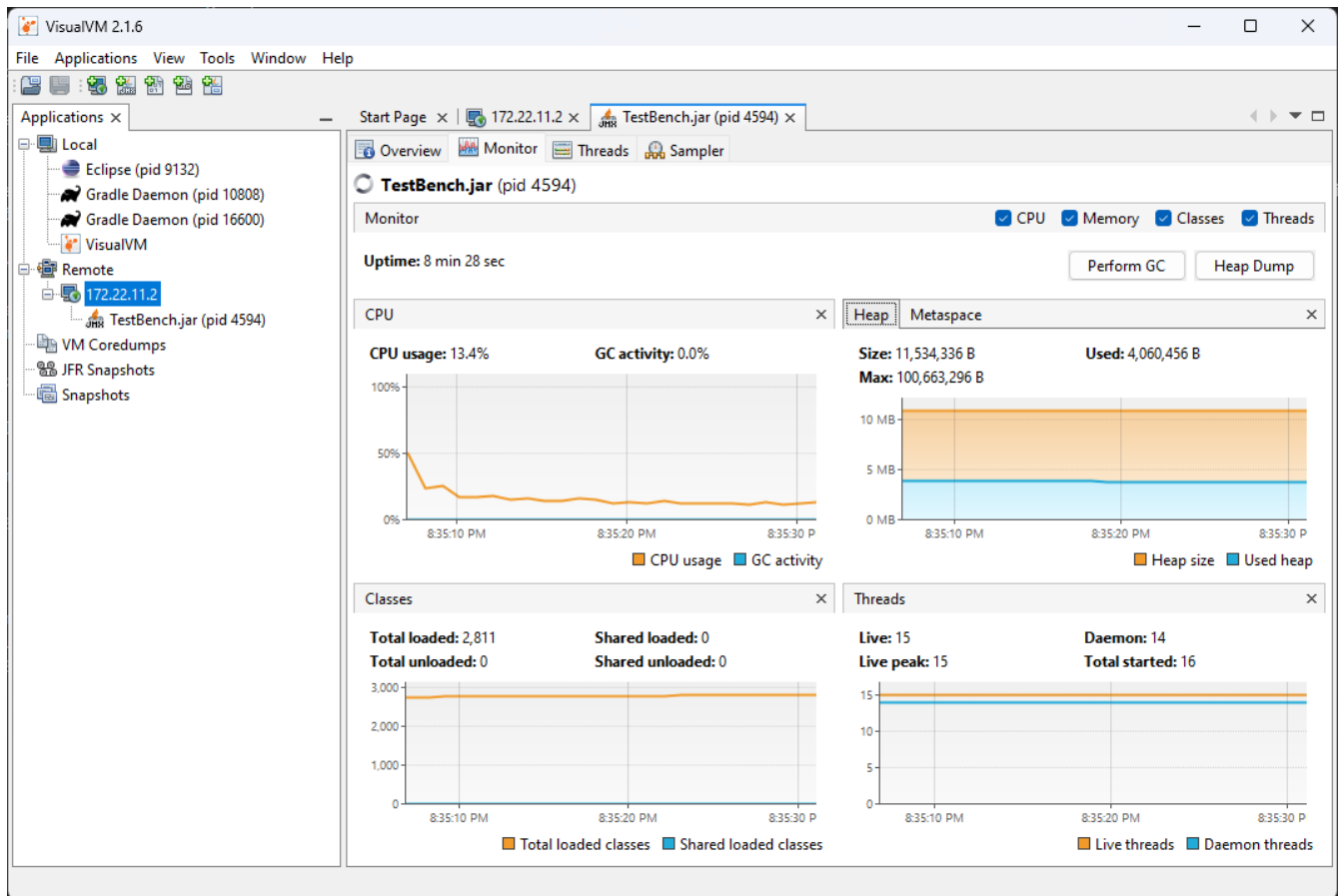
打开对话框后，配置连接详情和主机名。确保已勾选 Do not require SSL connection。



VisualVM connection dialog is ticked / 已勾选相应选项的 VisualVM 连接对话框

If correctly done, a new menu option in the left-hand sidebar will appear. Clicking on it will show you a detailed dashboard of the running JVM application.

如果操作正确，左侧边栏会出现一个新的菜单项。点击它，即可查看正在运行的 JVM 应用的详细监控面板。

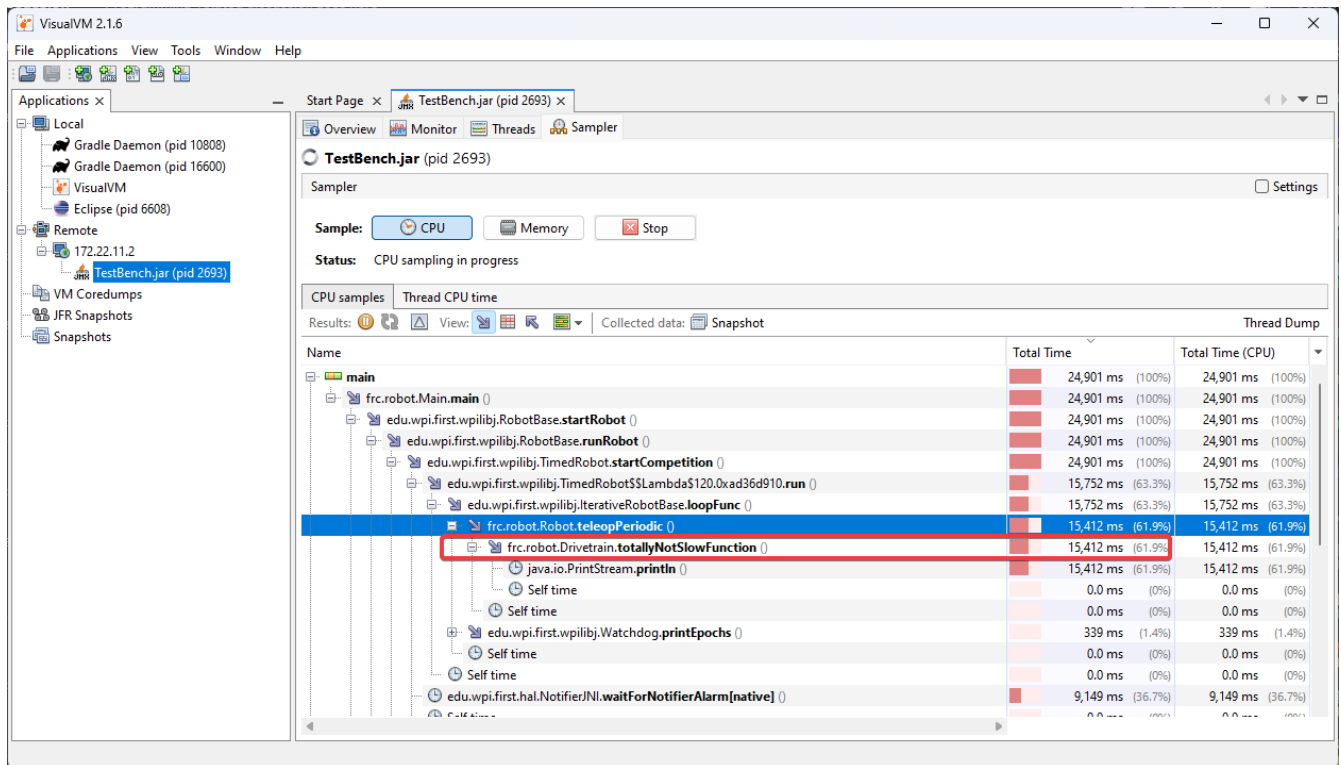


VisualVM diagnostics dashboard / VisualVM 诊断面板

Analyzing Function Timings / 分析函数耗时

An important feature of VisualVM is the ability to view how much time a specific function is taking up. This is *without* having a code debugger attached. To begin, click on the Sampler tab and then click on CPU. This will immediately give a breakdown of what functions are taking CPU time.

VisualVM 的一项重要功能，是查看某个函数耗费了多少时间，而且不需要连接代码调试器。首先，点击 Sampler 选项卡，再点击 CPU。这会立即显示各个函数占用 CPU 时间的情况。



Analyzing the VisualVM function timing tree / 分析 VisualVM 函数耗时树

The above screenshot shows a breakdown of the total time a specific function takes. You can see that `totallyNotSlowFunction()` accounts for 61.9% of the robot program CPU time. We can then correlate this to our robot program. In `totallyNotSlowFunction()`, we have the following code.

上方截图展示了某个函数总耗时的分解。可以看到，`totallyNotSlowFunction()` 占机器人程序 CPU 时间的 61.9%。随后可以将其对应到机器人程序。在 `totallyNotSlowFunction()` 中，代码如下。

```
public static void totallyNotSlowFunction() {
    for (int i = 0; i < 2000; i++) {
        System.out.println("HAHAHAHA");
    }
}
```

In this code snippet, we can identify 2 major causes of concern. A long running for loop blocks the rest of the robot program from running. Additionally, `System.out.println()` calls on the roboRIO are typically quite expensive. We found this information by profiling the Java application on the roboRIO!

在这段代码中，可以发现 2 个主要问题。长时间运行的 for 循环会阻止机器人程序的其他部分运行。此外，在 roboRIO 上调用 `System.out.println()` 通常开销较大。对 roboRIO 上的 Java 应用进行性能分析，让我们发现了这些信息！

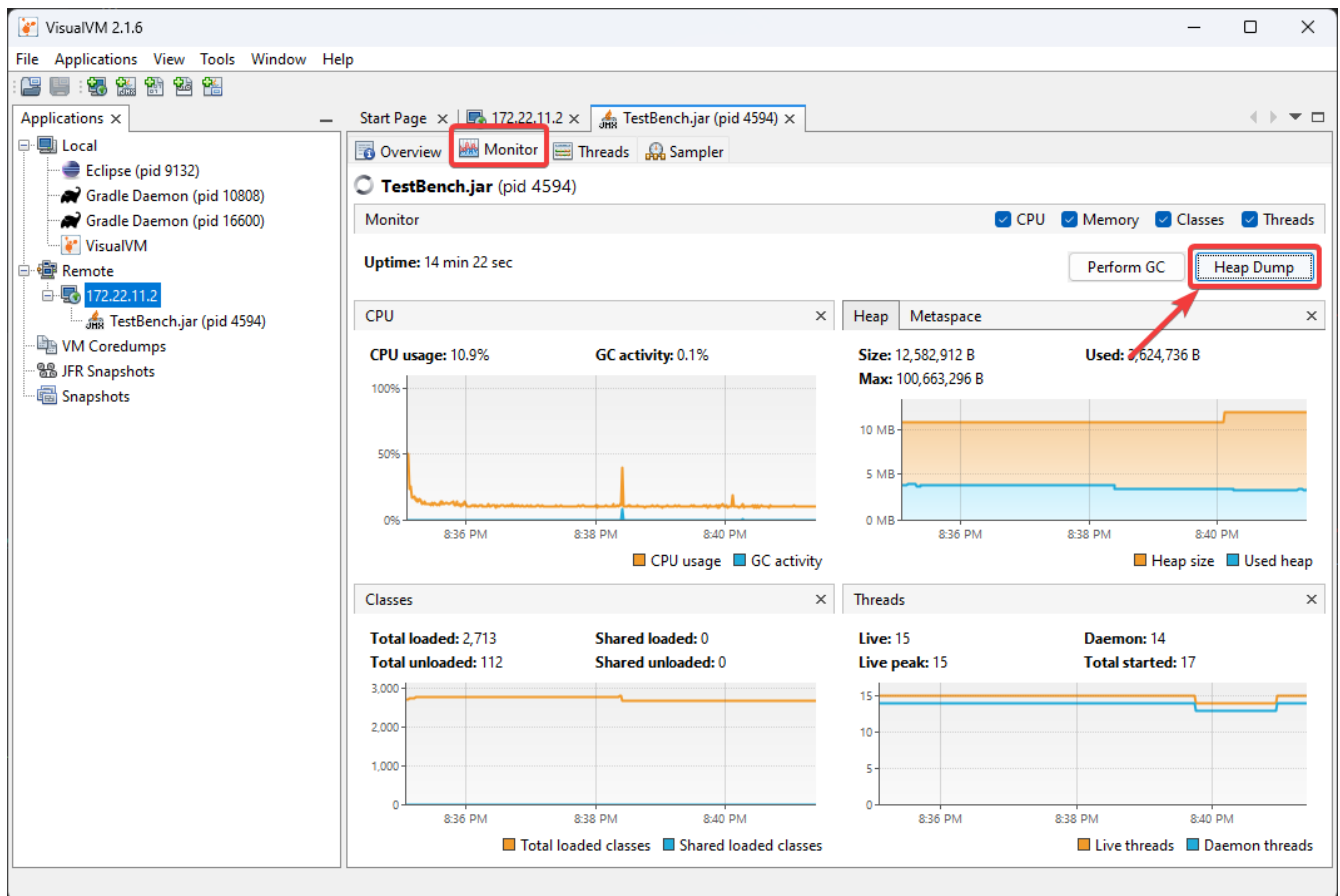
Creating a Heap Dump / 创建堆转储

Besides viewing the remote systems CPU and memory usage, VisualVM is most useful by creating a **Heap Dump**. When a Java object is created, it resides in an area of memory called the heap. When the heap is full, a process called [garbage collection](#) begins. Garbage collection can be a common cause of loop overruns in a traditional Java robot program.

除了查看远程系统的 CPU 和内存使用情况，VisualVM 最有用的功能是创建堆转储。创建 Java 对象时，对象会位于称为堆的内存区域中。当堆满时，称为[垃圾回收](#)的过程就会开始。垃圾回收可能是传统 Java 机器人程序循环超时的常见原因。

To begin, ensure you are on the Monitor tab and click Heap Dump.

首先，确保处于 Monitor 选项卡，然后点击 Heap Dump。



Location of heap dump button in VisualVM / VisualVM 中堆转储按钮的位置

This heap dump will be stored on the target system (roboRIO) and must be retrieved using SFTP. See [this article](#) for information on retrieving the dump from the roboRIO.

该堆转储会保存在目标系统（roboRIO）上，必须使用 SFTP 取回。有关从 roboRIO 取回转储的说明，请参阅[这篇文章](#)。

Once downloaded, the dump can be analyzed with VisualVM.

下载完成后，即可使用 VisualVM 分析转储。

Tip / 提示

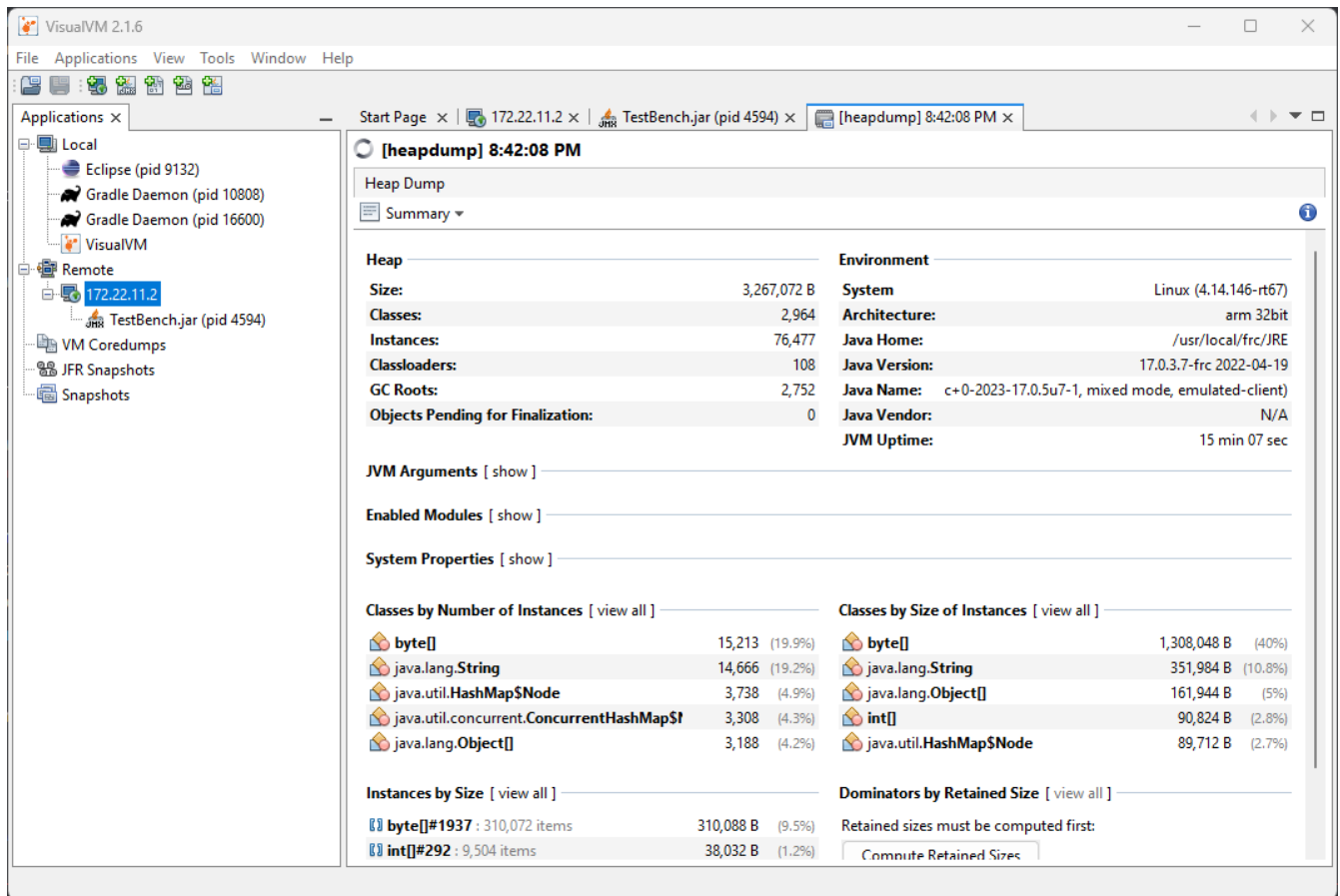
You can also configure the JVM to take a heap dump automatically when your robot code runs out of memory.

还可以配置 JVM，使其在机器人代码内存耗尽时自动生成堆转储。

Analyzing a Heap Dump / 分析堆转储

Reopen VisualVM if closed using the previous instructions. Then click on File and Load. Navigate to the retrieved dump file and load it.

如果已关闭 VisualVM，请按前面的说明重新打开。然后点击 File 和 Load，找到取回的转储文件并加载它。



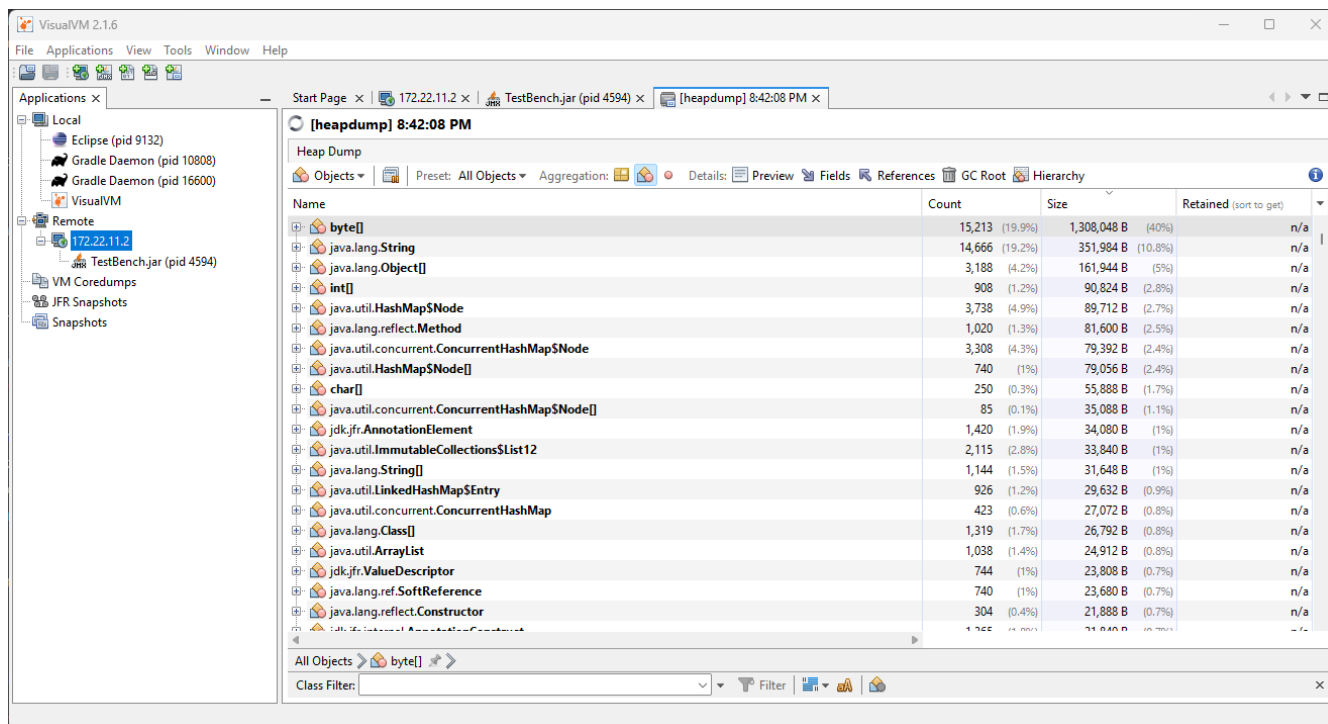
Viewing a dump in VisualVM / 在 VisualVM 中查看转储

Clicking on Summary and selecting Objects instead will show a breakdown of objects by quantity. The below screenshot showcases a completely empty robot program, and then one that creates an million large ArrayList of integers.

点击 Summary 后改为选择 Objects，即可按数量查看对象分布。下方截图展示了一个完全空白的机器人程序，以及一个创建了包含一百万个整数的大型 ArrayList 的程序。

Blank robot program:

空白机器人程序：



VisualVM 2.1.6

File Applications View Tools Window Help

Applications x

Local

- Eclipse (pid 9132)
- Gradle Daemon (pid 10808)
- Gradle Daemon (pid 16600)
- VisualVM

Remote

- 172.22.11.2
- TestBench.jar (pid 4594)
- VM Coredumps
- JFR Snapshots
- Snapshots

Start Page x | 172.22.11.2 x | TestBench.jar (pid 4594) x | [heapdump] 8:42:08 PM x

[heapdump] 8:42:08 PM

Heap Dump

Objects | Preset: All Objects | Aggregation: | Details: Preview Fields References GC Root Hierarchy

Name	Count	Size	Retained (sort to get)
byte[]	15,213 (19.9%)	1,308,048 B (40%)	n/a
java.lang.String	14,666 (19.2%)	351,984 B (10.8%)	n/a
java.lang.Object[]	3,188 (4.2%)	161,944 B (5%)	n/a
int[]	908 (1.2%)	90,824 B (2.8%)	n/a
java.util.HashMap\$Node	3,738 (4.9%)	89,712 B (2.7%)	n/a
java.lang.reflect.Method	1,020 (1.3%)	81,600 B (2.5%)	n/a
java.util.concurrent.ConcurrentHashMap\$Node	3,308 (4.3%)	79,392 B (2.4%)	n/a
java.util.HashMap\$Node[]	740 (1%)	79,056 B (2.4%)	n/a
char[]	250 (0.3%)	55,888 B (1.7%)	n/a
java.util.concurrent.ConcurrentHashMap\$Node[]	85 (0.1%)	35,088 B (1.1%)	n/a
jdk.jfr.AnnotationElement	1,420 (1.9%)	34,080 B (1%)	n/a
java.util.ImmutableCollections\$List12	2,115 (2.8%)	33,840 B (1%)	n/a
java.lang.String[]	1,144 (1.5%)	31,648 B (1%)	n/a
java.util.LinkedHashMap\$Entry	926 (1.2%)	29,632 B (0.9%)	n/a
java.util.concurrent.ConcurrentHashMap	423 (0.6%)	27,072 B (0.8%)	n/a
java.lang.Class[]	1,319 (1.7%)	26,792 B (0.8%)	n/a
java.util.ArrayList	1,038 (1.4%)	24,912 B (0.8%)	n/a
jdk.jfr.ValueDescriptor	744 (1%)	23,808 B (0.7%)	n/a
java.lang.ref.SoftReference	740 (1%)	23,680 B (0.7%)	n/a
java.lang.reflect.Constructor	304 (0.4%)	21,888 B (0.7%)	n/a

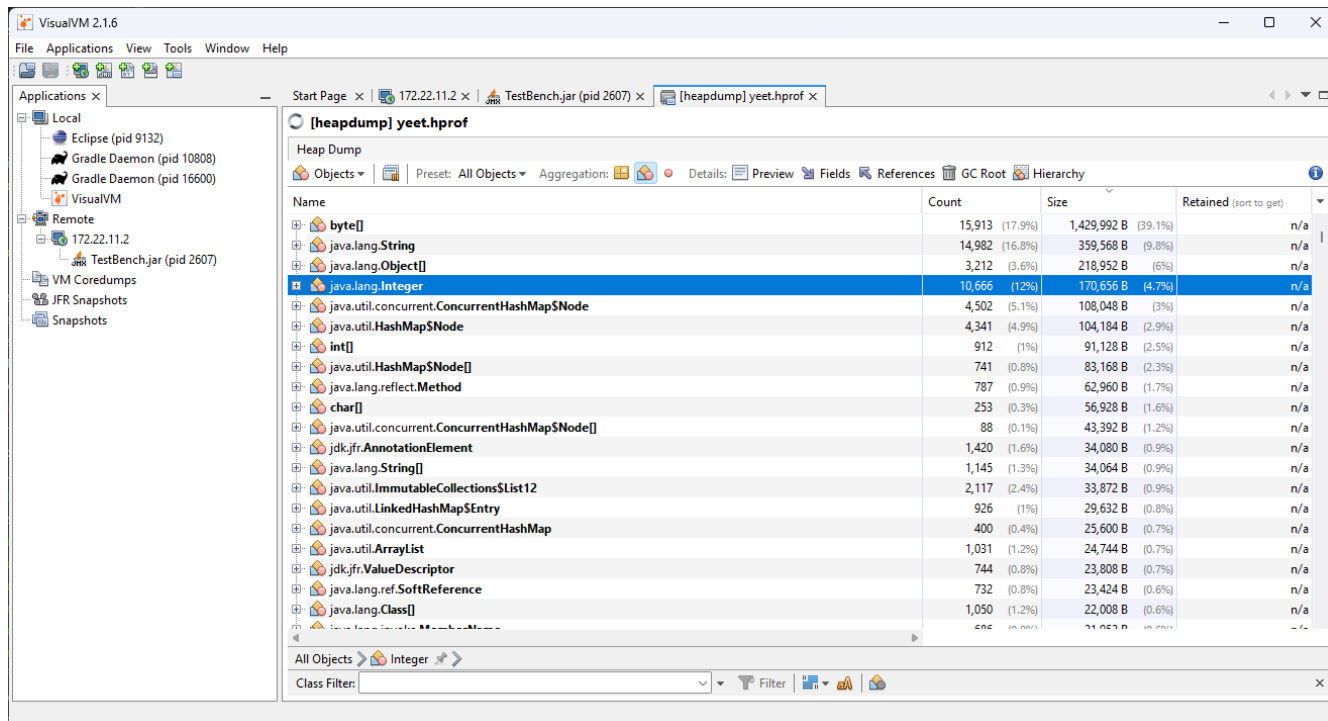
All Objects | byte[] | Filter

List of objects in a blank robot program / 空白机器人程序中的对象列表

with an ArrayList of ~10000 integers.

包含 ~10000 个整数的 ArrayList 的程序。

译注：原文前一段写“一百万个整数”，此处则写“~10000 个整数”，两处数量不一致；译文按原文保留。



VisualVM 2.1.6

File Applications View Tools Window Help

Applications x

Local

- Eclipse (pid 9132)
- Gradle Daemon (pid 10808)
- Gradle Daemon (pid 16600)
- VisualVM

Remote

- 172.22.11.2
- TestBench.jar (pid 2607)
- VM Coredumps
- JFR Snapshots
- Snapshots

Start Page x | 172.22.11.2 x | TestBench.jar (pid 2607) x | [heapdump] yeet.hprof x

[heapdump] yeet.hprof

Heap Dump

Objects | Preset: All Objects | Aggregation: | Details: Preview Fields References GC Root Hierarchy

Name	Count	Size	Retained (sort to get)
byte[]	15,913 (17.9%)	1,429,992 B (39.1%)	n/a
java.lang.String	14,982 (16.8%)	359,568 B (9.8%)	n/a
java.lang.Object[]	3,212 (3.6%)	218,952 B (6%)	n/a
java.lang.Integer	10,666 (12%)	170,656 B (4.7%)	n/a
java.util.concurrent.ConcurrentHashMap\$Node	4,502 (5.1%)	108,048 B (3%)	n/a
java.util.HashMap\$Node	4,341 (4.9%)	104,184 B (2.9%)	n/a
int[]	912 (1%)	91,128 B (2.5%)	n/a
java.util.HashMap\$Node[]	741 (0.8%)	83,168 B (2.3%)	n/a
java.lang.reflect.Method	787 (0.9%)	62,960 B (1.7%)	n/a
char[]	253 (0.3%)	56,928 B (1.6%)	n/a
java.util.concurrent.ConcurrentHashMap\$Node[]	88 (0.1%)	43,392 B (1.2%)	n/a
jdk.jfr.AnnotationElement	1,420 (1.6%)	34,080 B (0.9%)	n/a
java.lang.String[]	1,145 (1.3%)	34,064 B (0.9%)	n/a
java.util.ImmutableCollections\$List12	2,117 (2.4%)	33,872 B (0.9%)	n/a
java.util.LinkedHashMap\$Entry	926 (1%)	29,632 B (0.8%)	n/a
java.util.concurrent.ConcurrentHashMap	400 (0.4%)	25,600 B (0.7%)	n/a
java.util.ArrayList	1,031 (1.2%)	24,744 B (0.7%)	n/a
jdk.jfr.ValueDescriptor	744 (0.8%)	23,808 B (0.7%)	n/a
java.lang.ref.SoftReference	732 (0.8%)	23,424 B (0.6%)	n/a
java.lang.Class[]	1,050 (1.2%)	22,008 B (0.6%)	n/a

All Objects | Integer | Filter

List of objects in a modified robot program / 修改后的机器人程序中的对象列表

Additional Info / 更多信息

For more information on VisualVM, check out the [VisualVM documentation pages](#).

有关 VisualVM 的更多信息，请参阅 [VisualVM 文档页面](#)。

JVM Runtime error / JVM 运行时错误

Programs compiled with Microsoft Visual C++ (MSVC) require a MSVC runtime that is equal to or newer than the newest library being executed. WPILib is compiled with a recent version of MSVC.

使用 Microsoft Visual C++ (MSVC) 编译的程序，要求 MSVC 运行时版本不低于正在执行的最新库所要求的版本。WPILib 使用较新的 MSVC 版本编译。

In the past, this hasn't really been used, but a change in the runtime in 2024 actually took advantage of this requirement.

过去，这一点实际上并未被明显用到，但 2024 年运行时的一项变化真正依赖了这一要求。

Most JDKs ship with an MSVC runtime, and many of them are old. Additionally, many of them don't actually include the full runtime. Both of these are a problem, as all the JNI libraries loaded by WPILib both use a lot of the runtime, and are compiled against basically the newest runtime.

大多数 JDK 都附带 MSVC 运行时，其中许多版本较旧。此外，许多 JDK 实际上并未包含完整运行时。这两种情况都会造成问题，因为 WPILib 加载的所有 JNI 库既使用了大量运行时功能，又基本上都是针对最新运行时编译的。

For the JDK shipped with WPILib, the newest runtime is installed to ensure this issue isn't run into. But other JDKs will fail. So in 2025, much more important than prior years, the WPILib JDK needs to be used.

WPILib 随附的 JDK 会安装最新运行时，以确保不出现此问题。但其他 JDK 会失败。因此在 2025 年，使用 WPILib JDK 比往年重要得多。

The following error will be seen if a JDK with an incompatible MSVC runtime is used. This only affects Windows users.

如果使用的 JDK 附带不兼容的 MSVC 运行时，就会出现以下错误。此问题仅影响 Windows 用户。

```
> Task :simulateJavaRelease FAILED
If you receive errors loading the JNI dependencies, make sure you have the latest Visual Studio C++ Redistributable installed.
That can be found at https://support.microsoft.com/en-us/help/2977003/the-latest-supported-visual-c-downloads
Exception in thread "main" edu.wpi.first.util.MsvcRuntimeException: Invalid MSVC Runtime Detected.
Expected at least 14.40, but found 14.29
JVM Location: C:\Program Files\Amazon Corretto\jdk21.0.5_11\bin\java.exe
Runtime DLL Location: C:\Program Files\Amazon Corretto\jdk21.0.5_11\bin\msvcp140.dll
See https://wpilib.org/jvmruntime for more information

    at edu.wpi.first.util.WPIUtilJNI.checkMsvcRuntime(Native Method)
    at edu.wpi.first.wpilibj.RobotBase.startRobot(RobotBase.java:470)
    at frc.robot.Main.main(Main.java:23)

FAILURE: Build failed with an exception.
```

Setting Gradle to use WPILib JDK / 设置 Gradle 使用 WPILib JDK

This is not necessary if using the WPILib VS Code, which is the supported way to run. The following methods may be used for other Windows users.

如果使用 WPILib VS Code，则无需进行此设置；这是受支持的运行方式。其他 Windows 用户可以使用以下方法。

Ensure you've [installed WPILib](#) before proceeding.

继续之前，请确保已[安装 WPILib](#)。

Commandline Parameter / 命令行参数

To ensure that Gradle uses the WPILib JDK, you must set `org.gradle.java.home` to the WPILib JDK. This can be done by passing a parameter to gradle (where YEAR is the current WPILib installation):

要确保 Gradle 使用 WPILib JDK，必须将 `org.gradle.java.home` 设置为 WPILib JDK 的路径。可以通过向 Gradle 传递参数来完成，其中 YEAR 是当前 WPILib 安装所对应的年份：

```
gradlew -D org.gradle.java.home="C:\\Users\\Public\\wpilib\\YEAR\\jdk" ...
```

gradle.properties / gradle.properties

It can also be done by putting `gradle.properties` in the project root with the following contents (where YEAR is the current WPILib installation):

也可以在项目根目录中放置包含以下内容的 `gradle.properties` 文件，其中 YEAR 是当前 WPILib 安装所对应的年份：

Warning / 警告

`gradle.properties` should not be checked into git or other version management tools. This will only work if you only use your project on Windows, as the location is hardcoded to the location on Windows.

不应将 `gradle.properties` 提交到 Git 或其他版本管理工具中。此方法只有在项目仅用于 Windows 时才能正常工作，因为其中硬编码了 Windows 上的位置。

```
org.gradle.java.home=C:\\Users\\Public\\wpilib\\YEAR\\jdk
```

IDE / IDE

Some IDEs have a method to set the JDK used. Consult the documentation for your IDE.

某些 IDE 提供了设置所用 JDK 的方法。请参阅所用 IDE 的文档。